

```

/*before Stat make a absolute path name*/
strcpy (temppath,dir);
strcat (temppath, "/");
strcat (temppath, dp->d_name);
/*stat - populates the structure with the
attributes of the file/directory passed as the
first argument*/
stat (temppath,&sbuf);
/*Checking if it is a directory, you can
check many other things too */
if (S_ISDIR(sbuf.st_mode)) {
    /*Again call this function to
    walk down this directory*/
    traversedir (temppath);
}
}
}
closedir (dfd);
}
}

```

Now, if you compile this code and run it, it outputs the absolute pathnames of all directories starting from the current directory. The comments in the code will help you understand the basic algorithm.

You can consider modifying the code to add error-handling and more features. Please also read the manual pages for the functions used. You can experiment with these in your own code.

File access and manipulation: System calls vs library functions

Whenever you access a file, the OS (to be more specific, a system call) internally converts the filename to the inode number to get its attributes or data. The data structures used internally are called the user file descriptor table, the file table and the inode table. You need to remember that whenever your program opens a file, a file descriptor (a non-negative number) is returned to your program, which you can use to read/write the file's data. By default, three descriptors—0 (standard input), 1 (standard output) and 2 (standard error) are automatically allocated to every running program. When you open a file using library functions, instead of a file descriptor, you use a file pointer. A simple program will help you understand these concepts better, so here's a program to copy a file using library functions – it is called *mycpy_lib.c*:

```

#include <stdio.h>
#include <stdlib.h>

int main (int argc, char *argv []) {
    FILE *sfp,*dfp;
    int ch;

```

```

if (argc < 3) {
    printf ("Usage:mycpy sourcefile destinationfile\n");
    exit (1);
}
if ((dfp=fopen (argv[2],"w")) ==NULL) {
    printf ("Cannot open %s", argv[2]);
    exit (1);
}
if ((sfp = fopen(argv[1],"r"))==NULL) {
    printf ("Cannot open %s", argv[1]);
    exit (1);
}
while ((ch = fgetc ( sfp )) != EOF) {
    printf ("%c", (char)ch);
    fputc (ch, dfp);
}
fclose (sfp);
fclose (dfp);
return 0;
}

```

Compile (*gcc mycpy_lib.c -o mycpy*) and run it with two command-line arguments—an existing file, and a name for the copy. Then note the functions used, like *fopen*, *fgetc*, *fputc* and *fclose*. *fopen* returns a file pointer (to a structure named *FILE*), which is used as an argument to all the other functions. You can assume that for every open file in your process, there is an associated *FILE* structure. The function *fgetc* returns a character read from the file, while *fputc* writes a character to the file.

You can see that the *while* loop runs till the end of the file has been reached. How does the library function *fgetc* know that the file has ended? The code below, *mycpy_sys.c*, which uses system calls instead of library functions, will help answer these questions:

```

#include<stdio.h>
#include<fcntl.h>
#include<stdlib.h>
#include<errno.h>
#include<sys/types.h>
#include<sys/stat.h>
#include<errno.h>
#include<unistd.h>
#define BLOCKSIZE 100

int main (int argc, char *argv []) {
    int sfd, dfd, bytesread, ret = 0;
    char buf [BLOCKSIZE];
    if (argc < 3) {
        printf ("Usage:mycpy sourcefile
destinationfile\n");
        exit (1);
    }

```